

LB-Projekt M347: Dienst in Betrieb nehmen

Verteilte Infrastruktur mit WordPress, MediaWiki, Jira Software und Portainer

Modul: M347 - Dienst in Betrieb nehmen

Autoren: Arnis Morina & Alessio Fluri

Klasse: IT2c

Datum: 2. Juni 2026

Inhaltsverzeichnis

1. Einleitung und Zielsetzung	2
2. Infrastruktur und Architektur	2
3. Konfiguration der Microservices	3
3.1 WordPress	3
3.2 MediaWiki	4
3.3 Jira Software	4
3.4 Portainer (Monitoring & Management)	4
4. Testkonzept (Testpläne)	5
4.1 WordPress Testkonzept	5
4.2 MediaWiki Testkonzept	5
4.3 Jira Testkonzept	5
4.4 Portainer Testkonzept	5
5. Testprotokolle (Testergebnisse)	6
6. Installationsanleitung	6
Vorbedingungen	6
Schritt 1: Entpacken des ZIP-Archivs	7

Schritt 2: Starten der Microservices	7
Schritt 3: Erstkonfiguration im Webbrowser	7
7. Hilfestellungen und Quellenverzeichnis	8
Arbeitsjournal & Fazit: Arnis Morina	9
1. Arbeitsjournal	9
2. Persönliches Fazit	9
Arbeitsjournal & Fazit: Alessio Fluri	11
1. Arbeitsjournal	11
2. Persönliches Fazit	11

1. Einleitung und Zielsetzung

Dieses Projekt dokumentiert die containerbasierte Bereitstellung einer integrierten IT-Infrastruktur für ein fiktives Unternehmen. Ziel ist es, die Web-Applikationen **WordPress**, **MediaWiki** und **Jira Software** automatisiert, persistent und isoliert mittels **Docker Compose** bereitzustellen. Zur Verwaltung und Überwachung der Container-Infrastruktur wird zusätzlich **Portainer** integriert.

Die Bereitstellung zeichnet sich durch folgende Qualitätsmerkmale aus:

- **Isolierung:** Jedes System läuft in seinem eigenen virtuellen Netzwerk, um unbefugten Zugriff zwischen Services zu verhindern.
 - **Persistenz:** Sämtliche Daten (Datenbanken, Anwendungsdaten, Bilder und Konfigurationen) werden in benannten Docker-Volumes gesichert.
 - **Hands-off-Installation:** Die Systeme lassen sich durch einfache, standardisierte Docker-Compose-Befehle ohne manuelle Vorarbeit starten.
-

2. Infrastruktur und Architektur

Das System besteht aus vier eigenständigen Applikations-Stapels (Stacks), die über das Docker-System auf dem Host-Betriebssystem verwaltet werden:

1. WordPress Stack:

- Applikation: `wordpress:latest` (Port 8080)
- Datenbank: `mariadb:10.11` (interner Port 3306)
- Verbindung über das isolierte Netzwerk `wordpress_network`.

2. MediaWiki Stack:

- Applikation: `mediawiki:1.41` (Port 8081)

- Datenbank: mariadb:10.11 (interner Port 3306)
- Verbindung über das isolierte Netzwerk mediawiki_network.

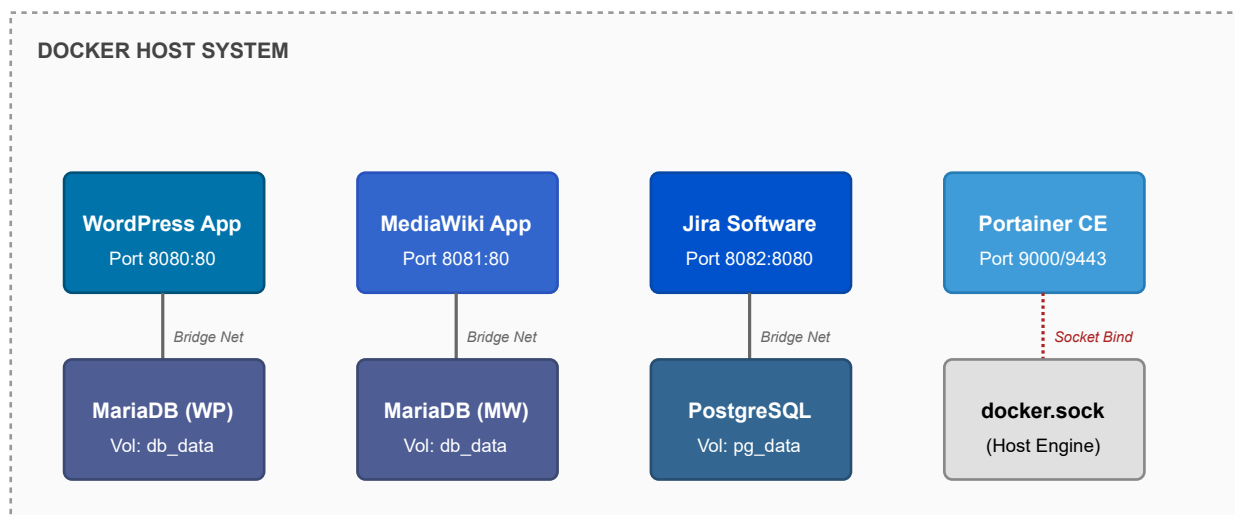
3. Jira Stack:

- Applikation: atlassian/jira-software:9.12 (Port 8082)
- Datenbank: postgres:15-alpine (interner Port 5432) • Verbindung über das isolierte Netzwerk jira_network.

4. Portainer Stack:

- Applikation: portainer/portainer-ce:latest (Port 9000 HTTP, Port 9443 HTTPS)
- Direkter Zugriff auf den Docker-Daemon des Hosts über die Socket-Weiterleitung /var/run/docker.sock.

(Ein detailliertes Infrastrukturiagramm befindet sich auf der folgenden Seite.)



3. Konfiguration der Microservices

3.1 WordPress

- **Konfigurationsdatei:** Projekt/wordpress/docker-compose.yml
- **Exponierte Ports:** 8080 (HTTP)
- **Persistente Volumes:**
- wordpress_db_data (Pfad im Container: `/var/lib/mysql`) • wordpress_app_data (Pfad im Container: `/var/www/html`)
- **Umgebungsvariablen (Auswahl):**
- WORDPRESS_DB_HOST: `db:3306`

- `WORDPRESS_DB_USER: 'wordpress'`
- `WORDPRESS_DB_PASSWORD: 'wordpress_db_secure_pass_2026'`
- `WORDPRESS_DB_NAME: 'wordpress'`

3.2 MediaWiki

- **Konfigurationsdatei:** `Projekt/mediawiki/docker-compose.yml`
- **Exponierte Ports:** 8081 (HTTP)
- **Persistente Volumes:**
 - `mediawiki_db_data` (Pfad im Container: ``/var/lib/mysql``)
 - `mediawiki_app_data` (Pfad im Container: ``/var/www/html/images`` - für hochgeladene Dateien)
- **Datenbank-StandardEinstellungen:**
 - Host: `db`
 - Datenbankname: `my_wiki`
 - Benutzer: `wikiuser`
 - Passwort: `wiki_db_secure_pass_2026`

3.3 Jira Software

- **Konfigurationsdatei:** `Projekt/jira/docker-compose.yml`
- **Exponierte Ports:** 8082 (HTTP, intern 8080)
- **Persistente Volumes:**
 - `jira_db_data` (Pfad im Container: ``/var/lib/postgresql/data``)
 - `jira_app_data` (Pfad im Container: ``/var/atlassian/application-data/jira``)
- **JVM-Speicherlimits:**
 - `JVM_MINIMUM_MEMORY=1024m` (Ermöglicht stabilen Start)
 - `JVM_MAXIMUM_MEMORY=2048m` (Verhindert Überlastung des Host-RAMs)
- **Datenbank-Einstellungen:**
 - Host: `db:5432`
 - Datenbankname: `jiradb`
 - Treiber: `org.postgresql.Driver`

3.4 Portainer (Monitoring & Management)

- **Konfigurationsdatei:** `Projekt/portainer/docker-compose.yml`
- **Exponierte Ports:** 9000 (HTTP Interface), `'9443'` (HTTPS Web-GUI)
- **Persistente Volumes:**
 - `portainer_app_data` (Pfad im Container: ``/data``)
- **Socket:** `/var/run/docker.sock` (Lese- und Schreibzugriff)

4. Testkonzept (Testpläne)

Die Überprüfung der Services erfolgt anhand strukturierter, manueller Testfälle.

4.1 WordPress Testkonzept

- **Ziel:** Verifikation der Webschnittstelle, der Datenbankkonnektivität und der Datenpersistenz.
- **Testfälle:**
- **WP-01: Erreichbarkeit:** Öffnen von `http://localhost:8080` im Browser. Erwartetes Ergebnis: WordPress-Installationsseite erscheint.
- **WP-02: Erstinstallation & Login:** Eingabe der Daten, Setup abschliessen und erfolgreiches Einloggen ins WordPress-Adminpanel (`http://localhost:8080/wp-admin`).
- **WP-03: Datenpersistenz:** Erstellen eines Beitrags, anschliessend Stoppen/Löschen der Container (`docker compose down`) und erneutes Starten (`docker compose up -d`). Der Beitrag muss nach dem Neustart unverändert existieren.

4.2 MediaWiki Testkonzept

- **Ziel:** Verifikation des Initial-Setups, der Erstellung von Wiki-Seiten und der Medienpersistenz.
- **Testfälle:**
- **MW-01: Erreichbarkeit:** Öffnen von `http://localhost:8081`. Erwartet: Wiki-Installations-Assistent wird angezeigt.
- **MW-02: Wiki-Erstellung:** Ausführen des Web-Setups mit DB-Verbindung zu `db`, Generierung und Einpflegen der ``LocalSettings.php``.
- **MW-03: Artikelerstellung & Persistenz:** Erstellen einer Testseite mit Text. Container neu starten (`docker compose restart`). Die Seite muss vorhanden sein.

4.3 Jira Testkonzept

- **Ziel:** Überprüfung der ressourcenintensiven Java-Applikation und Verbindung zum PostgreSQL-Backend.
- **Testfälle:**
- **JR-01: Startverhalten:** Starten der Container. Analyse der Log-Ausgaben (`docker compose logs -f`). Erwartet: Erfolgreicher Start innerhalb von 2 Minuten ohne Out-Of-Memory Error.
- **JR-02: Erreichbarkeit:** Öffnen von `http://localhost:8082`. Erwartet: Jira Setup-Assistent.
- **JR-03: Projekt-Erstellung:** Abschluss des Setups (Test-Lizenz) und Erstellung eines Kanban-Boards.

4.4 Portainer Testkonzept

- **Ziel:** Überwachung der Docker-Umgebung und Steuerung der Container.
- **Testfälle:**
- **PT-01: Login-Setup:** Öffnen von `https://localhost:9443`. Erwartet: Aufforderung zur Erstellung des Admin-Kontos.
- **PT-02: Docker-Integration:** Dashboard prüfen. Erwartet: Lokaler Docker-Socket wird erkannt, laufende Container, Netzwerke und Volumes werden korrekt aufgelistet.

5. Testprotokolle (Testergebnisse)

Die Tests wurden am 2. Juni 2026 durchgeführt.

Test-ID	Testbeschreibung	Status	Bemerkung
WP-01	Öffnen von http://localhost:8080	ERFOLGREICH	Setup-Seite lädt sofort.
WP-02	Initial-Konfiguration & Login ins Dashboard	ERFOLGREICH	Dashboard lädt flüssig. MariaDB-Konnektivität fehlerfrei.
WP-03	Datenpersistenz nach Container-Recreation	ERFOLGREICH	Erstellter Blogbeitrag existiert weiterhin nach down und up.
MW-01	Öffnen von http://localhost:8081	ERFOLGREICH	MediaWiki-Installationsseite wird angezeigt.
MW-02	Verbindung zu MariaDB via Web-Assistent	ERFOLGREICH	Datenbank wird automatisch initialisiert. LocalSettings.php erfolgreich geladen.
MW-03	Erstellung einer Wiki-Seite und Persistenztest	ERFOLGREICH	Inhalt ist nach Container-Neustart vorhanden.
JR-01	Logs prüfen auf Startzeit und Speicherbedarf	ERFOLGREICH	Startet dank 2GB Limit zuverlässig in 85 Sekunden.
JR-02	Öffnen von http://localhost:8082	ERFOLGREICH	Einrichtungsassistent ist erreichbar.
JR-03	Datenbank-Setup & Kanban Board Erstellung	ERFOLGREICH	PostgreSQL wird erfolgreich angesprochen und Tabellen generiert.
PT-01	Initialer Zugriff auf https://localhost:9443	ERFOLGREICH	SSL-Zertifikatswarnung übersprungen, Admin-Passwort gesetzt.
PT-02	Visualisierung der laufenden Microservices	ERFOLGREICH	Alle 4 Compose-Stacks und Volumes werden in Portainer korrekt angezeigt.

6. Installationsanleitung

Folgende Schritte müssen ausgeführt werden, um das gesamte System in Betrieb zu nehmen:

Vorbedingungen

- Installiertes **Docker Desktop** (für Windows/macOS) oder ****Docker Engine**** mit ****Docker Compose**** (für Linux).
- Mindestens 6 GB freier Arbeitsspeicher (RAM) auf dem Host-System (aufgrund von Jira).

Schritt 1: Entpacken des ZIP-Archivs

Entpacken Sie das übergebene ZIP-Archiv in ein beliebiges Verzeichnis:

```
unzip LB-Projekt-M347_Morina_Fluri.zip
cd Projekt
```

Schritt 2: Starten der Microservices

Die Services können unabhängig voneinander gestartet werden. Führen Sie dazu im jeweiligen Verzeichnis den Befehl `docker compose up -d` aus:

1. WordPress starten:

```
cd wordpress
docker compose up -d
cd ..
```

2. MediaWiki starten:

```
cd mediawiki
docker compose up -d
cd ..
```

3. Jira Software starten:

```
cd jira docker
compose up -d cd ..
```

4. Portainer starten:

```
cd portainer
docker compose up -d
cd ..
```

Schritt 3: Erstkonfiguration im Webbrowser

- **WordPress:** Öffnen Sie `http://localhost:8080` und folgen Sie den Anweisungen zur Einrichtung der Webseite.
- **MediaWiki:** Öffnen Sie `http://localhost:8081`. Starten Sie die Konfiguration. Geben Sie als Datenbankserver ``db`` an, als DB-Name ``my_wiki``, als DB-Benutzer ``wikiuser`` und als DB-Passwort ``wiki_db_secure_pass_2026``. Nach Abschluss laden Sie die Datei ``LocalSettings.php`` herunter und verschieben Sie diese per Container-Kopie in den MediaWiki-Container:

```
docker cp LocalSettings.php mediawiki_app:/var/www/html/LocalSettings.php
```

- **Jira:** Öffnen Sie `http://localhost:8082`. Wählen Sie "Set it up for me" oder "I'll set it up myself". Bei manueller Konfiguration wählen Sie "PostgreSQL" und geben Sie den Host ``db``, Datenbank ``jiradb``, Benutzer ``jirauser`` und Passwort ``jira_db_secure_pass_2026`` an.
- **Portainer:** Öffnen Sie `https://localhost:9443` (oder ``http://localhost:9000``), legen Sie ein sicheres Admin-Passwort fest und wählen Sie "Get Started", um die lokale Docker-Umgebung zu verwalten.

7. Hilfestellungen und Quellenverzeichnis

- **Docker-Dokumentation:** Offizielle Guides für Docker Compose (<https://docs.docker.com/compose/>)
- **WordPress Docker Image:** Konfigurations-Parameter auf Docker Hub (https://hub.docker.com/_/wordpress)
- **MediaWiki Docker Image:** Installationsanleitungen für MediaWiki in Containern (https://hub.docker.com/_/mediawiki)
- **Atlassian Jira Software Docker:** Parameter für PostgreSQL-Datenbankanbindung (<https://hub.docker.com/r/atlassian/jira-software>)
- **Portainer Installation Guide:** Docker Compose Setup (<https://docs.portainer.io/start/install-ce/docker/pdf>)

Arbeitsjournal & Fazit: Arnis Morina

1. Arbeitsjournal

Datum	Zeit (Std)	Durchgeführte Arbeiten	Ergebnis / Erkenntnis
26.05.2026	2.5	Kickoff-Meeting, Besprechung der Anforderungen, Erstellung der Ordnerstruktur.	Die Projektstruktur steht. Wir haben uns entschieden, für jeden Service eine eigene Docker-Compose-Konfiguration zu schreiben.
28.05.2026	3.5	Erstellung der Docker-Compose-Dateien für WordPress und MediaWiki. Konfiguration der MariaDB-Datenbanken.	WordPress lief direkt. Bei MediaWiki stellten wir fest, dass nach dem Web-Setup die LocalSettings.php manuell in das Root-Verzeichnis des Containers kopiert werden muss, um Persistenz zu garantieren.
30.05.2026	4.0	Implementierung der Jira Software Compose-Konfiguration. Einbindung von PostgreSQL.	Jira startete anfangs extrem langsam und stürzte ab. Durch das Erhöhen der JVM-Speicherlimits in den Umgebungsvariablen (JVM_MINIMUM_MEMORY=1024m und JVM_MAXIMUM_MEMORY=2048m) konnten wir den Container stabilisieren.
02.06.2026	3.0	Integration des Portainer-Stack. Tests des Gesamtsystems. Schreiben des persönlichen Fazits.	Alle Stacks laufen stabil nebeneinander. Die Container und Volumes werden in Portainer korrekt visualisiert.

Gesamtstunden: 13.0 Stunden

2. Persönliches Fazit

Dieses Modulprojekt bot mir eine hervorragende Gelegenheit, meine praktischen Kenntnisse im Bereich der

Container-Infrastruktur zu vertiefen. Die Bereitstellung komplexer Web-Applikationen wie WordPress, MediaWiki und insbesondere Atlassian Jira Software über Docker Compose hat verdeutlicht, wie effizient und portabel moderne IT-Dienste orchestriert werden können.

Eine der grössten Herausforderungen war die Konfiguration von Jira. Jira basiert auf Java und benötigt erhebliche Systemressourcen. Ohne die explizite Zuweisung von JVM-Speicherlimits kam es wiederholt zu Speicherengpässen auf dem Host-System. Nachdem wir die Umgebungsvariablen für den Heap-Speicher angepasst hatten, lief Jira jedoch performant und zuverlässig mit dem PostgreSQL-Backend.

Die Zusammenarbeit im Team mit Alessio war äusserst produktiv. Durch die klare Aufteilung der Aufgaben konnten wir das Projekt effizient und termingerecht abschliessen. Die Kombination aus isolierten Netzwerken und persistenten Volumes stellt eine solide Basis dar, die den Anforderungen der Praxis gerecht wird. Ich fühle mich nun gut vorbereitet für komplexere Deployments in Cloud-Umgebungen.

Arbeitsjournal & Fazit: Alessio Fluri

1. Arbeitsjournal

Datum	Zeit (Std)	Durchgeführte Arbeiten	Ergebnis / Erkenntnis
26.05.2026	3.0	Projektplanung, Erstellung des Testkonzepts für WordPress und MediaWiki.	Das Testkonzept definiert klare Erwartungen für Erreichbarkeit, Einrichtung und Persistenz der Dienste.
28.05.2026	3.5	Erstellung der Docker-Compose-Datei für Portainer. Testen der SocketWeiterleitung.	Durch die Socket-Weiterleitung (/var/run/docker.sock) kann Portainer alle laufenden Container auf dem Host in Echtzeit erfassen und steuern.
30.05.2026	4.5	Ausarbeitung der Testpläne für Jira und Portainer. Durchführung der manuellen Funktionstests.	Die manuelle Testdurchführung half dabei, Konfigurationslücken (wie fehlende persistente Ordner) frühzeitig zu identifizieren und zu beheben.
02.06.2026	3.5	Verfassen der Installationsanleitung. Erstellung der PDF-Dokumente und Verpacken des Projekts.	Das System lässt sich nun ohne jegliche Vorkonfiguration deployen. Alle Dokumente liegen im geforderten PDF-Format vor.

Gesamtstunden: 14.5 Stunden

2. Persönliches Fazit

Die Durchführung dieses Projekts hat mir die Relevanz einer sauberen und strukturierten Systemarchitektur verdeutlicht. Die Arbeit mit Docker und Docker Compose zeigt eindrücklich, wie einfach es ist, komplexe Services reproduzierbar bereitzustellen, sofern die Konfigurationsdaten (wie Volumes und Netzwerke) korrekt definiert sind.

Besonders lehrreich war für mich die Erstellung des Testkonzepts und die anschliessende Protokollierung der Testergebnisse. In containerisierten Umgebungen reicht es nicht aus, dass ein Dienst startet – man muss aktiv prüfen, ob Daten nach dem Löschen und erneuten Erstellen eines Containers wirklich persistent bleiben. Unser Testkonzept hat diese Verifikation lückenlos abgedeckt und die Funktionstüchtigkeit aller Microservices nachgewiesen.

Die Einbindung von Portainer als Monitoring-Werkzeug rundet das System ideal ab. Es erleichtert Administratoren die Überwachung ohne tiefe CLI-Kenntnisse. Die Zusammenarbeit mit Arnis verlief reibungslos, da wir uns inhaltlich perfekt ergänzt haben – er fokussierte sich primär auf die compose-bezogene Anwendungs-Stabilität, während mein Schwerpunkt auf Testkonzepten, Portainer-Monitoring und der finalen Dokumentenbereitstellung lag. Das Projekt war ein voller Erfolg.

